

Especificação Técnica de Arquitetura Enterprise: Plataforma de Streaming VOD de Doramas

1. Visão Geral e Arquitetura de Alto Nível

A plataforma é desenvolvida no modelo **SVOD (Subscription Video on Demand) de Alta Performance**, focada em conteúdo asiático (K-Dramas, C-Dramas, J-Dramas, T-Dramas). O modelo de negócios baseia-se em **Plano Único Full HD (1080p)** com controle estrito de acessos simultâneos por conta e expansão via **Add-on de Telas Adicionais**. A arquitetura adota o padrão **Event-Driven Microservices** e **Database-per-Service**, garantindo proteção contra compartilhamento indevido de contas, resiliência na ingestão automática de conteúdos via extração JSON/Proxy de mídia e entrega de vídeo com baixíssima latência.

Componentes de Infraestrutura

- **Frontend & Client:** Next.js (App Router), React Server Components, Tailwind CSS, Shaka Player / Video.js.
- **Edge Network & Gateway:** Cloudflare (WAF/DDoS Protection) e Kong API Gateway com rate-limiting e autenticação JWT.
- **Bancos de Dados Primários:**
 - **PostgreSQL (Neon / Supabase):** Dados relacionais do catálogo, assinaturas e transações financeiras.
 - **Redis Cluster:** Cache distribuído de sessão, controle do heartbeat de reprodução e bloqueio concorrente de telas.
- **Mensageria & Filas:** Apache Kafka e RabbitMQ para mensageria assíncrona do pipeline de vídeo e billing.
- **Storage & CDN:** AWS S3 / Cloudflare R2 integrado com Cloudflare Stream.

2. Regras de Negócio & Anti-Compartilhamento

2.1. Perfil Único por Conta (Vínculo por CPF/E-mail)

- **Eliminação de Perfis Secundários:** Cada conta possui apenas **1 perfil principal**. O fluxo de alteração de perfil foi removido para evitar o compartilhamento de credenciais entre múltiplos usuários.
- **Device Fingerprinting:** Cadastro automático de dispositivos baseados em *hash* combinando User-Agent, WebGL/Canvas Fingerprint e IP. Limite máximo de **5 dispositivos autorizados** cadastrados por conta.

2.2. Modelo de Precificação e Telas Simultâneas

- **Plano Base (1080p Full HD):** R\$ X/mês com direito a **1 tela simultânea**.
- **Add-on de Tela Extra:** +1 tela por +R\$ Y/mês (Limite máximo: **4 telas simultâneas** por conta).
- **Bloqueio Concorrente em Tempo Real (Redis Heartbeat Lock):**
 1. Ao dar PLAY, o cliente abre uma conexão WebSocket ou executa polling HTTP (/api/v1/playback/heartbeat) a cada **10 segundos**.
 2. O estado é persistido no Redis com TTL de 15 segundos:
account:{userId}:active_sessions.
 3. Caso o número de conexões ativas exceda 1 + extraScreensCount, o novo player é interrompido imediatamente com a mensagem: *"Limite de telas atingido. Encerre a reprodução em outro dispositivo ou adicione mais telas."*

3. Pipeline de Ingestão e Extração Automática de VODs (JSON Extractor)

O sistema conta com um worker de scraping/extração assíncrono capaz de autenticar em provedores de origem, interceptar respostas JSON e ingerir manifestos .m3u8 / .mp4 juntamente com legendas e metadados.

3.1. Contrato JSON do Extrator (Payload Normalizado)

Todo dado extraído é transformado para a seguinte estrutura antes de ser enviado para a API de Ingestão:

```
{
  "provider_id": "vod_provider_alpha",
  "extracted_at": "2026-09-11T11:52:47Z",
  "drama": {
    "title_portuguese": "Tudo Bem Não Ser Normal",
    "title_original": "Saikometeuri Geunyeosogae",
    "country_origin": "KR",
    "release_year": 2020,
    "synopsis": "Um drama sobre um agente de saúde comunitário e uma escritora com transtorno de personalidade antissocial...",
    "poster_url": "https://cdn.origem.com/posters/itsoo.jpg",
    "banner_url": "https://cdn.origem.com/banners/itsoo.jpg",
    "genres": ["Drama", "Romance", "Psicológico"]
  },
  "episodes": [
    {
      "episode_number": 1,
      "title": "Episódio 1: A Menina e o Sapato Vermelho",
      "duration_seconds": 4500,
      "manifest_url": "https://stream.origem.com/hls/ep1/index.m3u8",
      "format": "HLS",
      "headers_required": {
        "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36",
```

```

    "Referer": "https://player.origem.com/",
    "Origin": "https://player.origem.com"
  },
  "subtitles": [
    {
      "language": "pt-BR",
      "url": "https://cdn.origem.com/subs/ep1_ptbr.vtt"
    },
    {
      "language": "en-US",
      "url": "https://cdn.origem.com/subs/ep1_en.vtt"
    }
  ]
}
]
}
}

```

4. Arquitetura do Proxy de Mídia (Media Reverse Proxy)

Como os links extraídos geralmente exigem Headers específicos (Referer, User-Agent, Cookies de sessão da origem) para não serem bloqueados por *hotlinking*, o consumo **não é feito diretamente pelo player do cliente**.

Fluxo do Proxy:

1. O cliente faz uma requisição para a nossa API: GET `/api/v1/stream/ep_123/master.m3u8?token=HMAC_TOKEN`.
2. O servidor valida se o token é válido e se a conta do usuário não estourou o limite de telas.
3. O **Stream Proxy Service** busca o .m3u8 original na fonte, injetando os cabeçalhos (headersJson) salvos no banco de dados.
4. O servidor reescreve as URLs internas dos segmentos (.ts / .m4s) dentro do manifesto para apontarem de volta para o nosso proxy, entregando a mídia fluida sem expor a origem nem quebrar por CORS.

5. Modelagem do Banco de Dados Completa (Prisma ORM)

```

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

```

```

generator client {

```

```

    provider = "prisma-client-js"
}

enum SubscriptionStatus {
    ACTIVE
    PAST_DUE
    CANCELED
}

enum ContentCountry {
    KR // Coreia do Sul
    CN // China
    JP // Japão
    TW // Taiwan
    TH // Tailândia
}

// -----
// USER & IDENTITY (Sem múltiplos perfis)
// -----

model User {
    id          String          @id @default(uuid())
    email       String          @unique
    cpf         String          @unique
    passwordHash String
    fullName    String
    createdAt   DateTime        @default(now())
    updatedAt   DateTime        @updatedAt

    subscription      Subscription?
    registeredDevices Device[]
    watchProgresses   UserWatchProgress[]
    favoriteDramas    UserFavorite[]
}

model Device {
    id          String          @id @default(uuid())
    userId      String
    user        User            @relation(fields: [userId], references: [id],
onDelete: Cascade)
    fingerprint String          // Hash único do dispositivo
    deviceName  String          // Ex: Chrome 128 / Android TV
    lastIp      String
    lastSeenAt  DateTime        @updatedAt

    @@unique([userId, fingerprint])
}

```

```
// -----
// SUBSCRIPTION & BILLING (Plano 1080p + Add-on de Telas Extra)
// -----
```

```
model Subscription {
    id                String                @id @default(uuid())
    userId            String                @unique
    user              User                  @relation(fields:
[userId], references: [id], onDelete: Cascade)
    status            SubscriptionStatus @default(ACTIVE)
    baseQuality       String                @default("1080p")
    extraScreensCount Int                  @default(0) // Telas
adicionais compradas (Max: 3)
    stripeSubscriptionId String?            @unique
    stripeCustomerId  String?
    currentPeriodEnd  DateTime
    createdAt         DateTime              @default(now())
    updatedAt         DateTime              @updatedAt
}
```

```
// -----
// CATALOG & MEDIA ENGINE
// -----
```

```
model Drama {
    id                String                @id @default(uuid())
    titlePortuguese  String
    titleOriginal    String?
    synopsis         String                @db.Text
    countryOrigin     ContentCountry @default(KR)
    releaseYear       Int
    posterUrl        String
    bannerUrl        String
    rating            Decimal?             @db.Decimal(3, 1)
    createdAt         DateTime              @default(now())
    updatedAt         DateTime              @updatedAt

    genres            DramaGenre[]
    episodes          Episode[]
    favorites          UserFavorite[]

    @@index([countryOrigin])
}
```

```
model Genre {
    id    String    @id @default(uuid())
    name  String    @unique
}
```

```

    dramas DramaGenre []
}

model DramaGenre {
    dramaId String
    genreId String
    drama Drama @relation(fields: [dramaId], references: [id],
onDelete: Cascade)
    genre Genre @relation(fields: [genreId], references: [id],
onDelete: Cascade)

    @@id([dramaId, genreId])
}

model Episode {
    id String @id @default(uuid())
    dramaId String
    drama Drama @relation(fields: [dramaId],
references: [id], onDelete: Cascade)
    episodeNumber Int
    title String?
    manifestUrl String // URL .m3u8 / .mpd / .mp4
    extraIda
    headersJson Json? // Headers necessários para o
Proxy (Referer, Origin, Cookies)
    durationSeconds Int?
    createdAt DateTime @default(now())

    subtitles Subtitle[]
    watchProgresses UserWatchProgress[]

    @@unique([dramaId, episodeNumber])
}

model Subtitle {
    id String @id @default(uuid())
    episodeId String
    episode Episode @relation(fields: [episodeId], references: [id],
onDelete: Cascade)
    language String // pt-BR, en-US, ko-KR
    vttUrl String
}

// -----
// USER ENGAGEMENT & HISTORY
// -----

model UserWatchProgress {

```

```

    id                String    @id @default(uuid())
    userId            String
    user              User      @relation(fields: [userId], references:
[id], onDelete: Cascade)
    episodeId         String
    episode            Episode  @relation(fields: [episodeId], references:
[id], onDelete: Cascade)
    stoppedAtSeconds  Int
    isFinished         Boolean   @default(false)
    updatedAt          DateTime @updatedAt

    @@unique([userId, episodeId])
}

model UserFavorite {
    userId    String
    dramaId   String
    user      User      @relation(fields: [userId], references: [id],
onDelete: Cascade)
    drama     Drama     @relation(fields: [dramaId], references: [id],
onDelete: Cascade)
    createdAt DateTime @default(now())

    @@id([userId, dramaId])
}

```

6. Controladora de Telas Simultâneas em TypeScript (Redis Service)

```

import Redis from 'ioredis';

const redis = new Redis(process.env.REDIS_URL);

interface SessionCheckPayload {
    userId: string;
    deviceId: string;
    maxAllowedScreens: number; // 1 (Base) + extraScreensCount
}

export async function validatePlaybackSession({
    userId,
    deviceId,
    maxAllowedScreens,
}: SessionCheckPayload): Promise<{ allowed: boolean; activeStreams:
number }> {
    const redisKey = `account:${userId}:active_sessions`;

```

```

// Registrar heartbeat do dispositivo atual com TTL de 15 segundos
await redis.hset(redisKey, deviceId, Date.now().toString());
await redis.expire(redisKey, 15);

// Limpar sessões expiradas (>15 segundos atrás)
const allSessions = await redis.hgetall(redisKey);
const now = Date.now();
let activeCount = 0;

for (const [devId, timestamp] of Object.entries(allSessions)) {
  if (now - parseInt(timestamp, 10) > 15000) {
    await redis.hdel(redisKey, devId);
  } else {
    activeCount++;
  }
}

// Verificar se o limite de telas simultâneas foi atingido
if (activeCount > maxAllowedScreens) {
  return { allowed: false, activeStreams: activeCount };
}

return { allowed: true, activeStreams: activeCount };
}

```